RecruitO · Learning Series (v2)

Same-origin api.js, the lobby/room split & password reset

The browser side, updated for today: the single-origin api.js with the session-expiry redirect, the new forgot/reset pages, and the interview flow split into a consent lobby and a separate LiveKit meeting room with icon controls.

Updated study guide · reflects the project as of 27 October 2026

1. How the frontend is built

- **No framework** — plain HTML/CSS/JS, one page per screen, split into ES modules that all import api.js.
- **Single origin** — the page and the API share one origin; the browser calls `/api/...` and nginx proxies it. No cross-origin setup.
- **nginx** adds the CSP and `Cache-Control: no-cache` so code/style updates reach users on the next reload.

> **Why same-origin matters**
> It made the Cloudflare tunnel work with one link, removed CORS entirely, let cookies stay SameSite=Lax, and ended the per-host CSP/LAN-IP juggling — the site behaves the same on localhost, a LAN IP, and the tunnel.

2. api.js — the HTTP layer

2.1 Finding the API (now trivial)

```
function getApiBase() {
    const override = window.RECRUITO_CONFIG && window.RECRUITO_CONFIG.apiBaseUrl;
    if (override) return override.replace(/\/$/, "");
    return `${window.location.origin}/api`;   // same origin, proxied by nginx
}
```

2.2 request() + the session-expiry redirect (line by line)

```
let hasAuthenticatedSession = false, redirectingToSignIn = false;
async function request(path, options = {}) {
    const response = await fetch(getApiBase()+path, { ...options, credentials:"include" });
    if (response.ok && !isPublicPath(path)) hasAuthenticatedSession = true;
    if (!response.ok) {
        const body = await response.clone().json().catch(()=>null);
        if (response.status === 401 && hasAuthenticatedSession
            && !path.startsWith("/auth/login")) redirectToSignIn();  // index.html?auth=signin&expired=1
        throw new ApiError(response.status, body?.detail);
    }
    return response.status === 204 ? null : await response.json();
}
```

Line	What it does
`credentials:"include"`	Sends the session cookie with every call (same origin, so it always travels).
`hasAuthenticatedSession`	Set true once any protected call succeeds — proof we were logged in.
`401 + hasAuthenticatedSession`	A later 401 means the session expired mid-use → redirect to sign in with ?expired=1. Never fires on the login call or before any login, so the sign-in page can't loop.
`new ApiError`	Turns non-OK responses into typed errors the pages show to the user.

New api methods this round: `forgotPassword(email)`, `resetPassword(token, pw)`, and `getLivekitToken(interviewId)`.

3. The pages (updated map)

Page	Role
index.html (app.js)	Home / sign-in / manager workspace. Now has a 'Forgot password?' link and sidebar links to the profile and create-interviewer pages.
forgot-password.html / reset-password.html	NEW: request a reset email; set a new password from the emailed link.
public-jobs.html	Candidate side: browse roles (title + seniority + Blogger link only), apply, track status (no scores shown).
manager-applications.html	Rankings, the per-application schedule form (date/time + interviewer), and the floating assistant.
interview.html (interview-lobby.js)	NEW ROLE: the consent LOBBY — shows the interview details + consent checkboxes, then Join opens the room.
meeting-room.html (meeting-room.js)	NEW: the actual video room — LiveKit video, icon controls, live transcript.

4. The interview flow: lobby → meeting room (NEW)

> **Two pages now**
> A dashboard 'Join' opens **interview.html** (the lobby). There the participant ticks consent (transcription, and biometric for candidates). Clicking **Join** navigates to a separate page, **meeting-room.html**, carrying the consent choices in the URL; that page auto-connects the video.

```
// interview-lobby.js: validate the link, then hand off to the room with the consent flags
joinButton.addEventListener("click", () => {
    const tx  = transcriptConsent.checked ? "1" : "0";
    const bio = identityConsent.checked   ? "1" : "0";
    location.assign(`./meeting-room.html?token=${token}&tx=${tx}&bio=${bio}`);
});
```

The meeting room reads `tx`/`bio` from the URL, validates the token, then auto-joins: it calls getUserMedia, fetches a LiveKit token, publishes the camera+mic to LiveKit, subscribes to the other participant, and — if tx=1 — starts transcription.

4.1 The transcription recorder (the fix that made it work)

```
// record ONLY the audio track (the camera stream also has video, which an
// audio-only format rejects); each ~3.5s window is a complete, decodable file.
const audioStream = new MediaStream(localStream.getAudioTracks());
const beginSegment = () => {
    const recorder = new MediaRecorder(audioStream, { mimeType });
    recorder.addEventListener("dataavailable", async (e) => {
        if (e.data.size && socket.readyState === WebSocket.OPEN)
            socket.send(await e.data.arrayBuffer());   // one complete audio file
        beginSegment();                                // then the next segment
    });
    recorder.start();
    setTimeout(() => recorder.state === "recording" && recorder.stop(), 3500);
};
```

Line	What it does
`audioStream`	The camera+mic stream has a video track; an audio-only mimeType rejects it, so the recorder never started. Recording only the audio track fixed it.

Line	What it does
`start -> stop -> restart`	A timeslice only puts the file header on the first chunk; start/stop per segment makes every chunk a complete, decodable file.
`socket.send(arrayBuffer)`	Each complete clip is streamed to the backend, which transcodes + transcribes it with AssemblyAI.

4.2 Icon controls (NEW)

The mute, camera, and end-call buttons are now round **icon buttons** (inline SVG). Toggling mute/camera swaps the icon to a slashed 'off' version and tints the button; the end-call button is a red hang-up icon.

5. Delivery: Cloudflare + the gatekeepers

Candidates reach the app on the public Cloudflare URL (`...trycloudflare.com`). Because everything is one origin, the old three gatekeepers collapse to essentially one:

Gatekeeper	Now
Session cookie	Still proves identity; same-origin so it always travels (SameSite=Lax).
CORS	No longer needed — the page and API share an origin.
CSP connect-src	'self' (+ https:/wss: for LiveKit). No per-host tuning.

> **If a call fails now**
> With one origin, 'Could not reach the API' almost always means the API container is down or restarting (cloudflared briefly can't resolve 'frontend'), or an expired session (which now auto-redirects to sign in). The quick-tunnel URL also changes on restart — re-read it from the cloudflared logs.

End of Part 2. Tools (Part 0), backend (Part 1), frontend (Part 2) — all current as of 27 Oct 2026.